

Neurosymbolic Program Synthesis: From Enumerative Search to LLM-Guided Methods

Vanshika Singla

4/13/2026

```
function calculateDaysBetweenDates(begin, end){  
  ??  
}
```

20 Years of Research

```
function calculateDaysBetweenDates(begin, end) {  
  var beginDate = new Date(begin);  
  var endDate = new Date(end);  
  var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));  
  return days;  
}
```

What is Program Synthesis?

- Automatically generating a program from a description of what it should do

Input:		Output:	
john smith	→	J. Smith	J. Smith
jane doe	→	J. Doe	J. Doe
bob jones	→	B. Jones	B. Jones

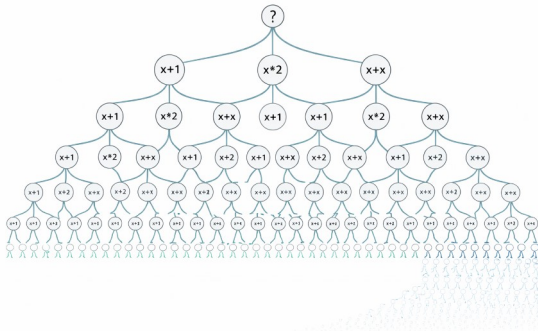


Synthesized program:

```
first_initial(name) = upper(name[0]) + "." + last(name)
```

Why Is It Hard?

1. The search space is enormous



Even simple examples are consistent with infinitely many programs

2. User intent is ambiguous

Examples:

$f(2) = 4$
 $f(3) = 6$

All three programs satisfy these examples

Program A:

$f(x) = x * 2$

$f(10) = 20$

Program B:

$f(x) = x + x$

$f(10) = 20$

Program C:

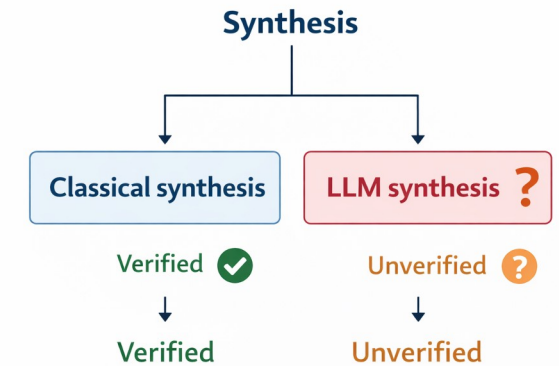
$f(x) = x + 2$

$f(10) = 12$

Same examples ($f(2)=4$, $f(3)=6$) but different behavior on a new input ($x=10$)

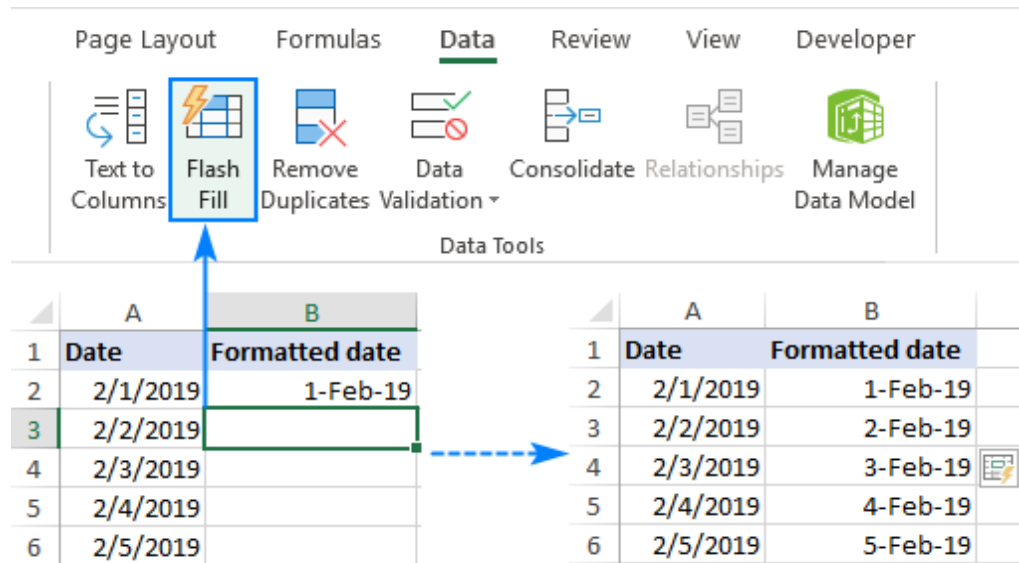
Which one did the user actually want?

3. Correctness is hard to guarantee



Neural methods are powerful but offer no formal guarantees

Why Does It Matter?



Excel FlashFill
ships to 1 billion+ Office users



GitHub Copilot
used by 1.3 million developers

Program synthesis is not just a research idea. It is already in tools you use every day.

Roadmap

- **Introduction:** what is synthesis and why does it matter
- **Solar-Lezama et al. (2006) — SKETCH:** correctness-guaranteed synthesis without domain rules
- **Gulwani (2011) — FlashFill:** synthesis that reached hundreds of millions of users
- **Gulwani, Polozov, Singh (2017) — Survey:** mapping the landscape
- **Ellis et al. (2021) — DreamCoder:** synthesis that learns over time 
- **Chen et al. (2021) — Codex:** LLMs enter the picture
- **Grand et al. (2024) — LILO:** combining classical and neural
- **Open problems and conclusion**

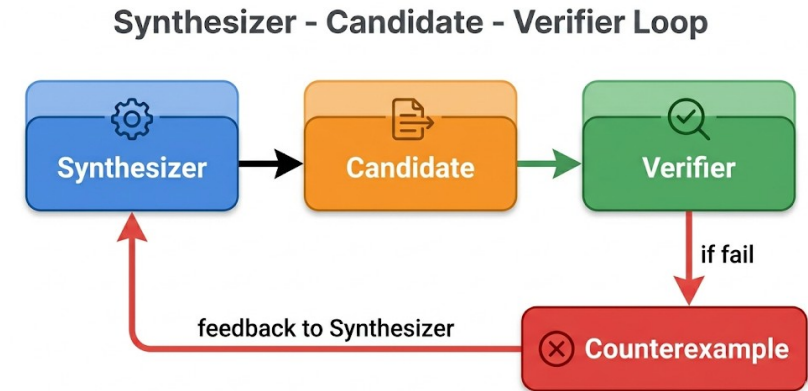
SKETCH (2006): Synthesis Without Domain Rules

Problem:

- Every synthesis tool before SKETCH needed hand-coded domain rules
- No general approach existed

Solution:

- Programmer writes partial program with holes
- SAT-based search fills the holes
- Correctness guaranteed by construction
- Introduced CEGIS (Counterexample Guided Inductive Synthesis):
generate candidate → test → counterexample → refine



SKETCH: Evaluation + Limitations

Evaluation:

- Synthesized complete AES encryption implementation
- SKETCH: 21.3ms vs OpenSSL: 19.6ms: less than 10% difference
- Original spec: 19,936ms: SKETCH was 1000x faster
- MultiGrid kernel: completed in under 5 minutes

Limitations:

- Programmer must understand algorithm to write a meaningful sketch
- Only works for finite programs: unbounded loops out of scope
- More holes = longer synthesis time

took 665 iterations to fill in

```
int[4] roundSK(bit[32][4] in, bit[32][4] rkey)
    implements round{
        int[4][256] T = ??;
        int[4] output = 0;
        bit[32] mask = 0x000000FF;
        int[4][4] ch = {{0,1,2,3},{1,2,3,0},
                        {2,3,0,1},{3,0,1,2}};
        for(int i=0; i<4; ++i){
            int i0 = (int) in[ch[i][0]] & mask;
            int i1 = (int)(in[ch[i][1]] >> 8) & mask;
            int i2 = (int)(in[ch[i][2]] >> 16) & mask;
            int i3 = (int)(in[ch[i][3]] >> 24) & mask;
            output[i] = T[0][i0] ^ T[1][i1]
                        ^ T[2][i2] ^ T[3][i3];
            output[i] = output[i] ^ rkey[i];
        }
        return output;
    }
```

Figure 5. Sketch for one round of AES.

SKETCH proved correctness-guaranteed synthesis was possible.

Next question: can it work without the programmer writing the sketch at all?

FlashFill: Synthesis for Real Users

Problem + Solution:

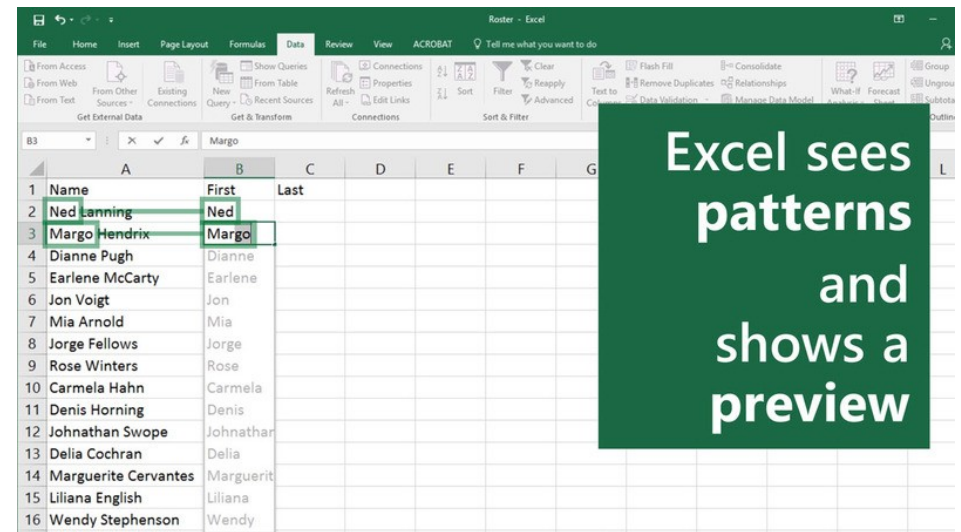
- Non-programmers reformatting spreadsheets constantly
- DSL for strings + version space algebras
- Runs in under one second while you type
- Example: "John Smith" → "J. Smith", "Jane Doe" → "J. Doe"

Evaluation:

- 175 benchmarks, 74% correct from single example
- Shipped in Excel 2013, hundreds of millions of users
- Most influential POPL paper award 2021

Limitations:

- Strings only, version space does not transfer to other domains
- Ambiguous examples can silently produce wrong program



FlashFill proved synthesis could reach real users. But it only worked in one narrow domain. The field needed a map of what existed and where to go next.

Survey (2017): Mapping the Landscape

Problem:

- Field fragmented across PL, ML, HCI with no shared vocabulary
- Researchers talking past each other

Three-axes framework:

- Intent: how does the user say what they want?
- Program space: what programs are possible?
- Search: how do you find the right one?

Coverage:

- Manna and Waldinger 1970s through early neural methods
- Four decades, three research communities

Limitations:

- Written just as neural methods got serious, coverage thin

Flags neurosymbolic combination as the major open direction

The survey identified the gap. DreamCoder filled it.

User Intent	Program Space	Search Strategy
• logical specs • examples • natural language • partial programs	• grammars • type systems • learned models	• enumerative • constraint solving • deductive • neural

DreamCoder (2021): Synthesis That Learns

Problem:

- Classical synthesis starts from scratch every time, does not accumulate knowledge
- Neural synthesis generalizes but lacks correctness guarantees
- Can a system get better at synthesis the more it synthesizes?

Key idea:

- Library learning: build up reusable abstractions over time instead of searching from scratch
- Neural policy guides search, gets smarter each iteration
- Wake-sleep algorithm: alternates between solving tasks and reflecting on what was learned

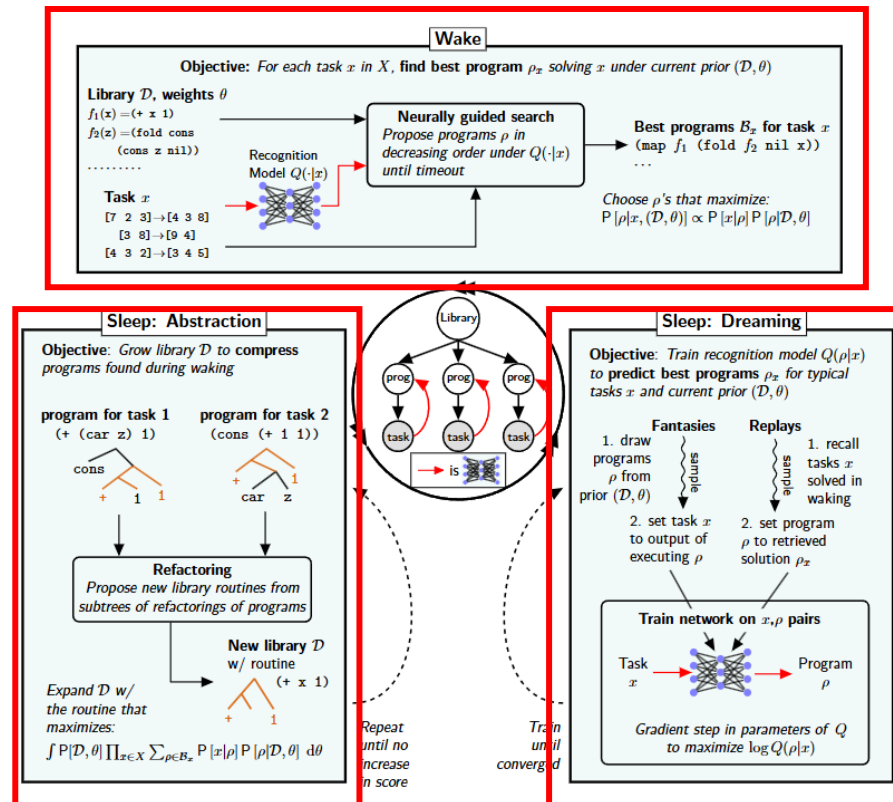
DreamCoder: The Wake-Sleep Algorithm

Wake Phase:

- Solve batch of tasks using neurally-guided enumerative search
- Recognition network scores which programs to explore first

Sleep Phase:

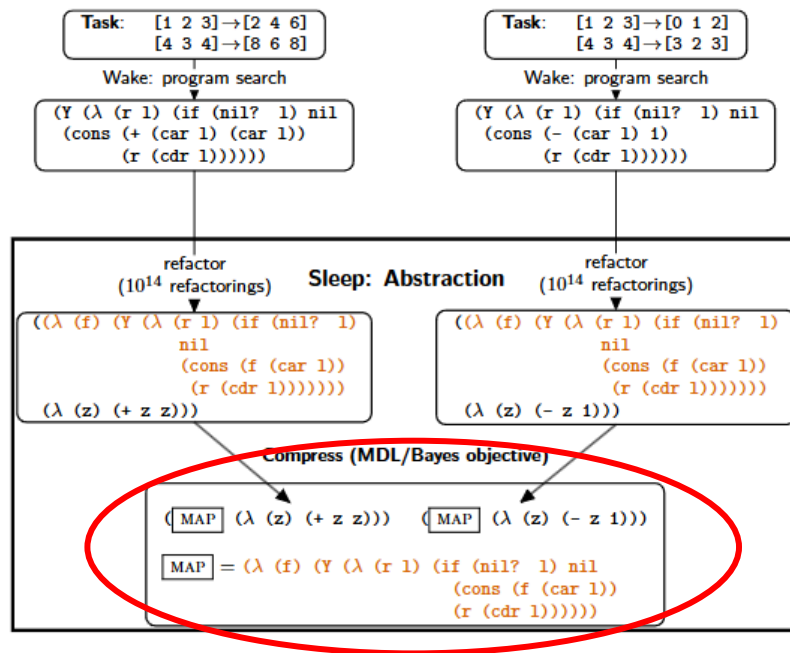
- Compress solutions, find shared substructures
- Add common patterns to library as named primitives
- Retrain neural policy on updated vocabulary



Each iteration starts from a richer language and a smarter guide

DreamCoder: Results

- Rediscovered map, filter, fold from basic recursion alone
- Discovered Newton's and Coulomb's laws from physics data
- Outperformed pure enumeration, pure neural, prior library learning across all 8 domains



Input	MAP program	Samples
(210) (220) (41) (635) (38)	Full (dd(d)*) No Library (dd(d)*. No Rec (dd(d)*)	(220) (461) (14u (2040) (68) (308)
\$5.70 \$3.40 \$2.80 \$5.40 \$3.70	Full \$.d0 No Library \$.d0 No Rec \$(d)*.(d)*0	\$2.40 \$3.30 \$5=50 \$7#40 \$.0 \$873.30
(715) 967-2697 (608) 819-2220 (920) 988-2524 (608) 442-0253 (262) 723-4043	Full (ddd) ddd-dddd No Library .ddd ddd,dddd No Rec .(d)* (d)*dd.(d)*	(099) 242-2029 (948) 452-9842 ?773) 726-6866 m627) 674,0602 z40192) 51(8 =2) 279-876273

This is not a benchmark improvement. This is a qualitatively different kind of result.

DreamCoder: Limitations and What Comes Next

Limitations:

- Enumerative search is expensive across 8 domains over many iterations
- Needs many training tasks to bootstrap library learning effectively
- Distribution shift between training and test hurts library transfer
- Cannot start from scratch in a new domain without significant setup

Codex (2021): When LLMs Entered the Picture

The question: Do you even need classical synthesis machinery with a large enough LLM?

Solution:

- GPT-3 fine-tuned on publicly available GitHub code
- Natural language prompt or docstring → token prediction → candidate program
- No search, no DSL, no specification language
- Became foundation for GitHub Copilot

```
def incr_list(l: list):  
    """Return list with elements incremented by 1.  
    >>> incr_list([1, 2, 3])  
    [2, 3, 4]  
    >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])  
    [6, 4, 6, 3, 4, 4, 10, 1, 124]  
    """  
    return [i + 1 for i in l]
```

White = prompt given by user.

Yellow = code generated by Codex.

Codex: Evaluation + Limitations

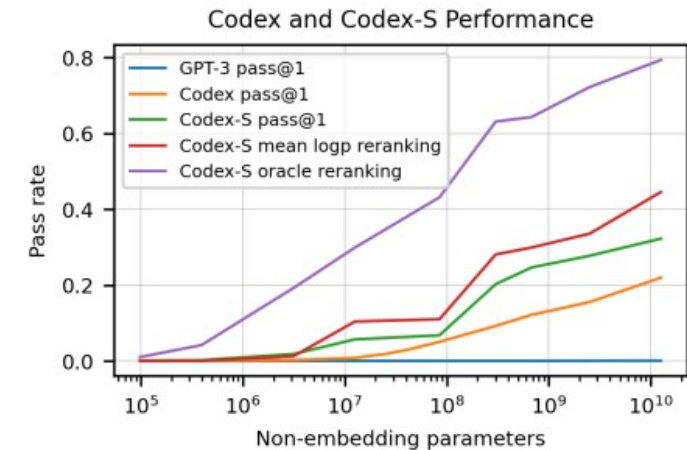
Evaluation:

- HumanEval: 164 hand-written Python problems with unit tests
- 28.8% pass@1: correct on first try
- 70.2% pass@100: correct within 100 samples
- Significantly exceeded all prior language models on code tasks
- Reset practical expectations across the entire field

Limitations:

- No notion of specification, no correctness guarantee
- LLMs predict likely tokens, synthesis requires provably correct programs: structurally different problems
- Fails on novel algorithms, edge cases, precise reasoning
- Code looks plausible but can fail silently on untested inputs

Fast and flexible but no formal guarantees. LILO asks: what if you keep the LLM speed but add back the classical structure?



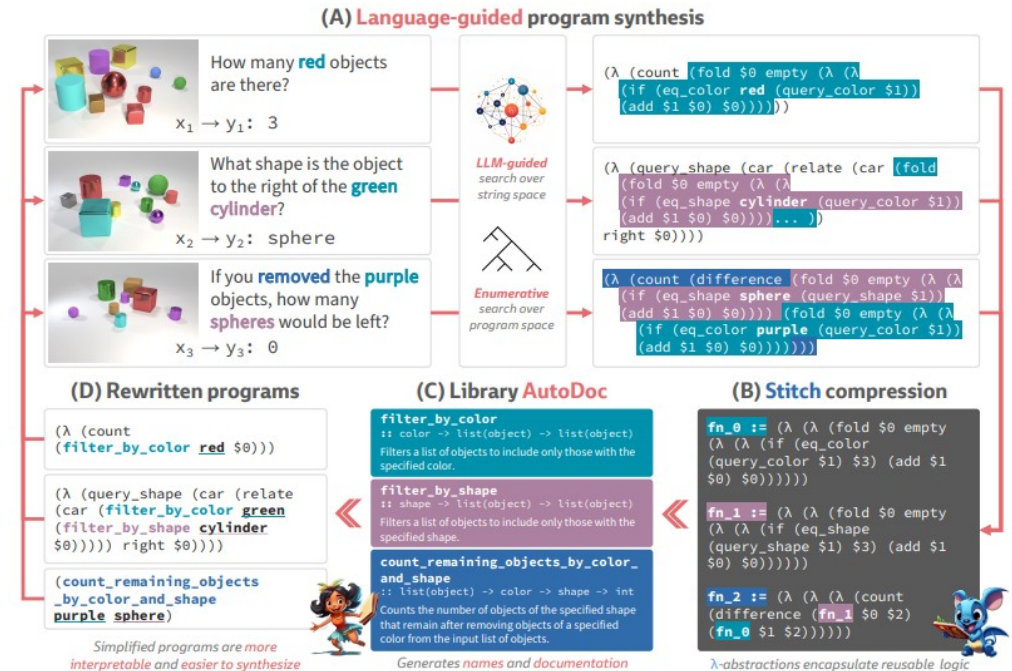
LILO (2024): The Best of Both Worlds

Problem:

- DreamCoder: powerful library learning but enumerative search is slow
- Codex: fast generation but no accumulated knowledge, no correctness
- Can we get LLM speed without giving up classical structure?

Solution:

- Replaces enumerative search in wake phase with LLM-guided synthesis
- Sleep phase keeps DreamCoder's compression and abstraction unchanged
- LLM generates candidates fast, classical machinery verifies and compresses
- Novel addition: auto-documentation generates readable names for learned functions



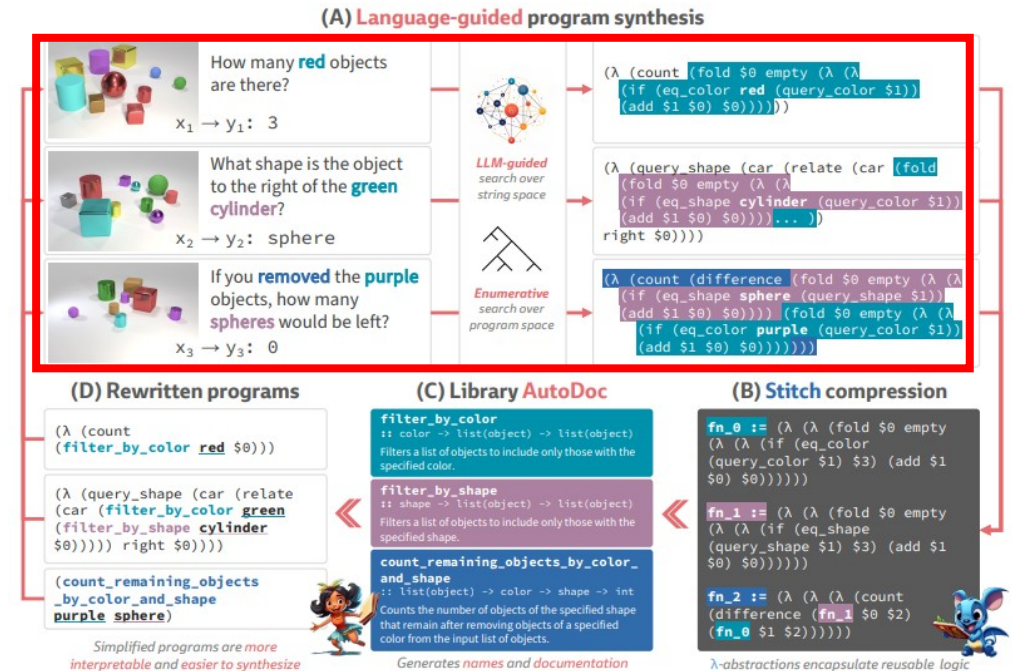
LILO (2024): The Best of Both Worlds

Problem:

- DreamCoder: powerful library learning but enumerative search is slow
- Codex: fast generation but no accumulated knowledge, no correctness
- Can we get LLM speed without giving up classical structure?

Solution:

- Replaces enumerative search in wake phase with LLM-guided synthesis
- Sleep phase keeps DreamCoder's compression and abstraction unchanged
- LLM generates candidates fast, classical machinery verifies and compresses
- Novel addition: auto-documentation generates readable names for learned functions



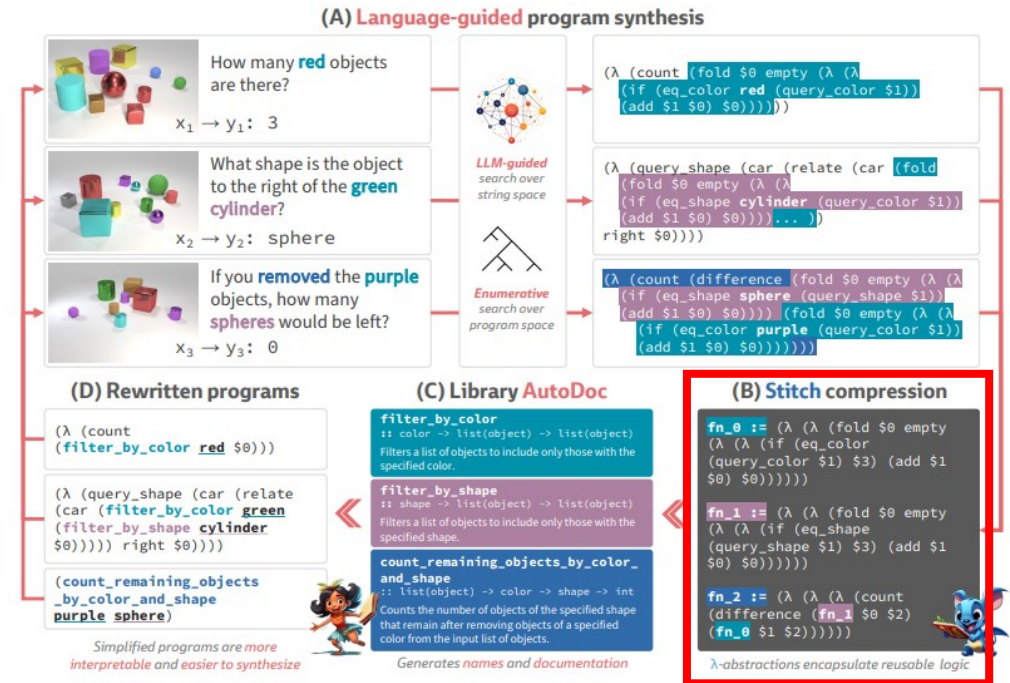
LILO (2024): The Best of Both Worlds

Problem:

- DreamCoder: powerful library learning but enumerative search is slow
- Codex: fast generation but no accumulated knowledge, no correctness
- Can we get LLM speed without giving up classical structure?

Solution:

- Replaces enumerative search in wake phase with LLM-guided synthesis
- Sleep phase keeps DreamCoder's compression and abstraction unchanged
- LLM generates candidates fast, classical machinery verifies and compresses
- Novel addition: auto-documentation generates readable names for learned functions



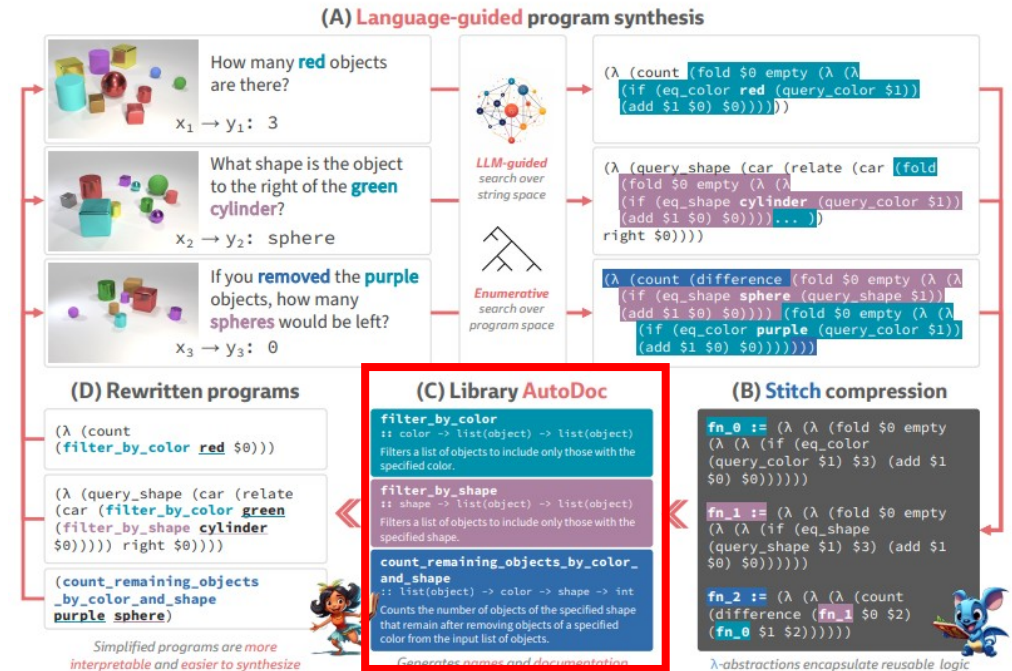
LILO (2024): The Best of Both Worlds

Problem:

- DreamCoder: powerful library learning but enumerative search is slow
- Codex: fast generation but no accumulated knowledge, no correctness
- Can we get LLM speed without giving up classical structure?

Solution:

- Replaces enumerative search in wake phase with LLM-guided synthesis
- Sleep phase keeps DreamCoder's compression and abstraction unchanged
- LLM generates candidates fast, classical machinery verifies and compresses
- Novel addition: auto-documentation generates readable names for learned functions



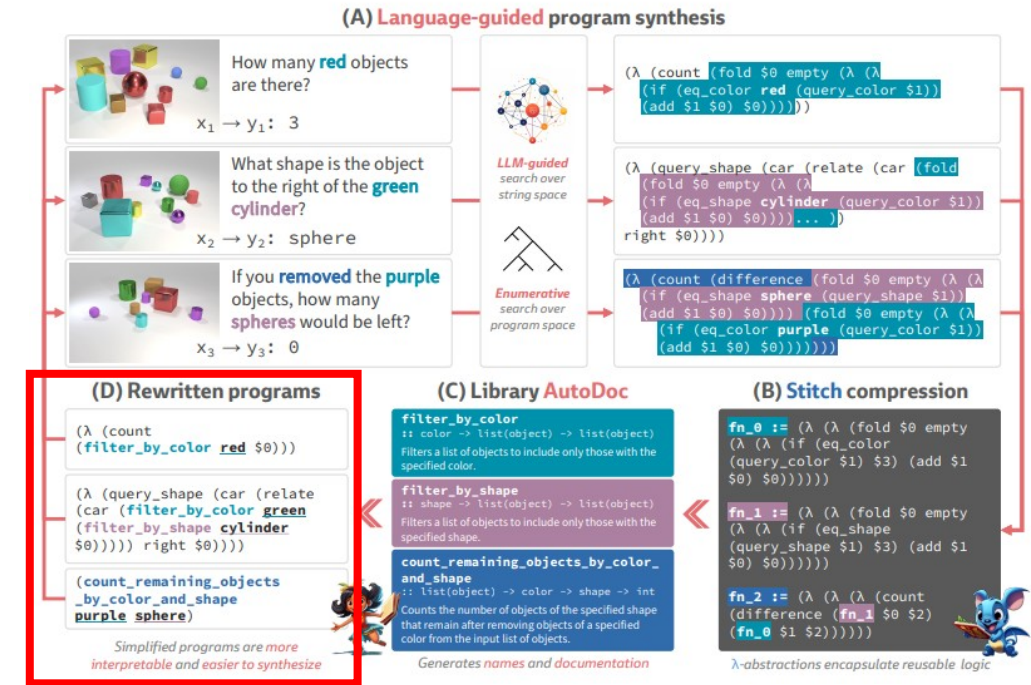
LILO (2024): The Best of Both Worlds

Problem:

- DreamCoder: powerful library learning but enumerative search is slow
- Codex: fast generation but no accumulated knowledge, no correctness
- Can we get LLM speed without giving up classical structure?

Solution:

- Replaces enumerative search in wake phase with LLM-guided synthesis
- Sleep phase keeps DreamCoder's compression and abstraction unchanged
- LLM generates candidates fast, classical machinery verifies and compresses
- Novel addition: auto-documentation generates readable names for learned functions



LILO: The Auto-Documentation Insight

The surprising finding:

- Giving LLM unnamed compressed abstractions actually hurt performance
- Without auto-documentation: solve rates dropped 30+ points on REGEX domain
- With auto-documentation: LLM generates readable names and docstrings for learned functions
- LLM reuses functions it can understand semantically, ignores ones it cannot

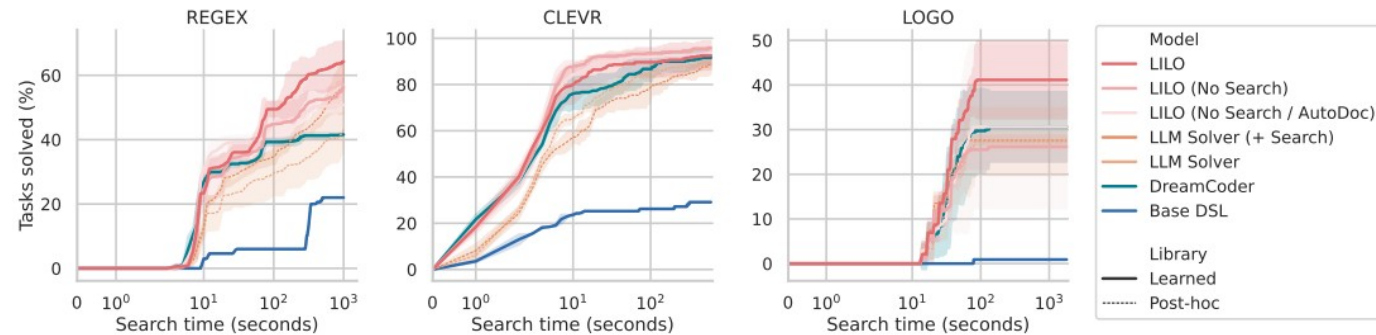
Example:

- Without doc: `fn_42 = (λ f. (λ z. (+ z z)))`
- With doc: `double = (λ f. (λ z. (+ z z))) : "Returns twice the input"`

LILO: Results + Limitations

Results:

- Outperformed DreamCoder by 33.14 points on REGEX
- Outperformed DreamCoder by 20.42 points on LOGO
- Outperformed DreamCoder by 2.26 points on CLEVR (where DreamCoder was already at 94.5%)
- Improved over LLM-only baselines by up to 16.82 points on LOGO



Limitations:

- Still requires hand-designed domain primitives to initialize library
- Cannot bootstrap from scratch in a genuinely new domain
- Depends on capable LLM, introduces cost and latency at scale

Open Problems: Where Does the Field Go Next?

- Classical synthesis: correct but only works in narrow domains with hand-designed DSLs
- Neural synthesis: flexible and fast but fails silently with no formal guarantees
- Neurosymbolic synthesis: promising but expensive and only tested on three domains

- How do you handle genuinely ambiguous intent?
- How do you build tools non-experts can actually trust?
- Can formal guarantees survive as systems get more neural?
- Can we build a general synthesizer without a hand-designed DSL?

- Recent work: hybrid LLM plus formal synthesis approaches now solving 80% of benchmarks — but a truly general solution is still out of reach

Conclusion

- Classical methods: solved correctness, not generality
- Neural methods: solved generality, not correctness
- Neurosymbolic methods: the attempt to solve both
- SKETCH made synthesis domain-agnostic
- FlashFill made it practical for real users
- Survey mapped the landscape and identified the gap
- DreamCoder made it cumulative
- Codex made it fast but unverified
- LILO combined both, best results so far
- Progress: from synthesizing AES in an hour (2006) to GitHub Copilot (2021) in twenty years
- What remains: a general synthesis system that works anywhere without hand-engineering does not exist yet

Can computers write programs for us? The field is closer than ever. But it is not there yet.

References

- [1] M. Chen et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.
- [2] K. Ellis et al. "DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning." PLDI, 2021.
- [3] G. Grand et al. "LILO: Learning Interpretable Libraries by Compressing and Documenting Code." ICLR, 2024.
- [4] S. Gulwani. "Automating String Processing in Spreadsheets Using Input-Output Examples." POPL, 2011.
- [5] S. Gulwani, O. Polozov, R. Singh. "Program Synthesis." Foundations and Trends in Programming Languages, 2017.
- [6] A. Solar-Lezama et al. "Combinatorial Sketching for Finite Programs." ASPLOS, 2006.

Thank You!

Questions?